INSTITUTO FEDERAL
Rio Grande do Sul
Campus Erechim

AULA 04 - HERANÇA – Generalização e especialização

Programação Orientada a Objetos I

Dário L. Beutler
dario.beutler@erechim.ifrs.edu.br

Objetivo da aula

- Entender conceitos ligados a HERANÇA na programação OO

Programa da aula

1) Hierarquia de Classes
2) Generalização
3) Especialização

Reuso/Reutilização

- Para entregar software de qualidade em menos tempo, é preciso reutilizar;
  - "Copiar & colar" não é reuso!
- Reuso é uma das principais vantagens anunciadas pela Orientação a Objetos;
- Mecanismo baseado no conceito de classes:
  - Composição ("tem um");
  - Herança ou derivação ("é um").

Composição

- Criação de uma nova classe usando classes existentes como atributos;
- Relacionamento "tem um": uma conta tem um titular (cliente), um cliente tem um nome (*string*);

Herança

- Criação de novas classes derivando classes existentes;
- Relacionamento "é um [subtipo de]": um livro é um produto, um administrador é um usuário;
- Uso da palavra-chave `extends`;
- A palavra-chave é sugestiva – a classe que está sendo criada "estende" outra classe:
  - Partindo do que já existe naquela classe...
  - Pode adicionar novos recursos;
  - Pode redefinir recursos existentes.

Motivação

- Classes com elementos (atributos, métodos) repetidos:

```
class Produto {
  String nome;
  double preco;

  Produto() { } // Precisa?

  public Produto(String nome, double preco) {
     this.nome = nome; this.preco = preco;
  }

  public boolean ehCaro() {
     return (preco > 100);
  }

  // Eventuais outros métodos...
}
```

Motivação

- Classes com elementos (atributos, métodos) repetidos:

```
class Livro {
  String nome;
  double preco;
  String autor;
  int paginas;

  public Livro(String n, double p, String a, int pg) {
     nome = n; preco = p; autor = a; paginas = pg;
  }

  public boolean ehCaro() { return (preco > 100); }

  public boolean ehGrande() { return (paginas > 200); }

  // Eventuais outros métodos...
}
```

Não escreva assim, é só pra caber no slide!

Motivação

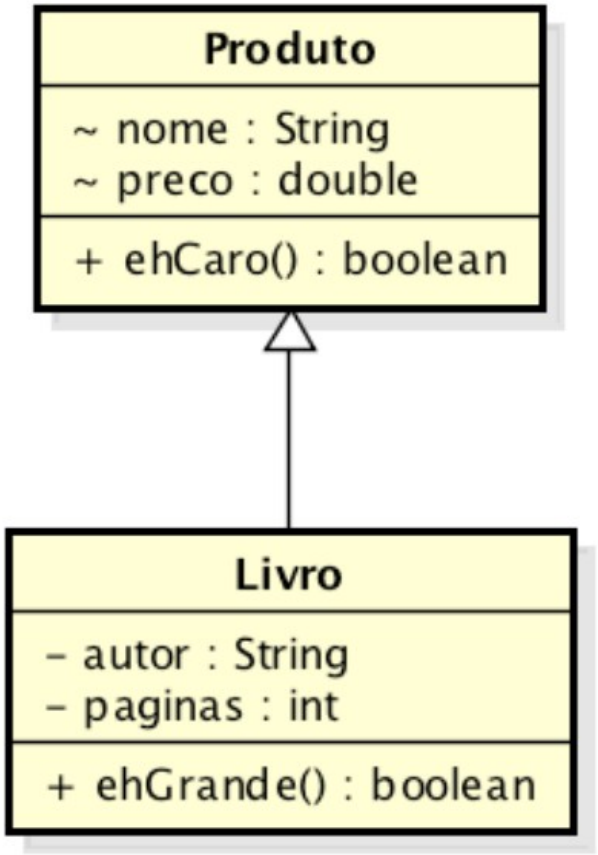
- Código repetido = problema de manutenção;
  - Se surge um novo tipo de produto?
  - Se muda alguma coisa em todos os produtos?
- Colocar os atributos extras em Produto, porém só utilizá-los em objetos que representem livros?
  - Solução confusa, desperdiça memória, ainda mais se a hierarquia crescer (CDs, DVDs, eletrônicos, etc.);
- Usar composição?
  - Também causa confusão. Um livro tem um produto ou um livro é um produto?
- Solução OO: herança!

Solução com herança

- Livro estende produto (adiciona novos membros):

```
class Livro extends Produto {
  //private String nome;        // Não preciso repetir.
  //private double preco;       // Herdo de Produto
  private String autor;
  private int paginas;

  public Livro(String n, double p, String a, int pg) {
     nome = n; preco = p; autor = a; paginas = pg;
  }

  // Também não preciso repetir:
  // public boolean ehCaro() { return (preco > 100); }

  public boolean ehGrande() { return (paginas > 200); }

  // Eventuais outros métodos...
}
```

Herança em UML



Um livro é um (tipo de) produto.

Solução com herança

- Podemos chamar métodos do Produto no Livro:

```
public class Loja {
  public static void main(String[] args) {
    Livro l = new Livro("Linguagens de Programação",
         74.90, "Flávio Varejão", 334);

    System.out.println(l.ehCaro());
    System.out.println(l.ehGrande());
  }
}
```

Produto

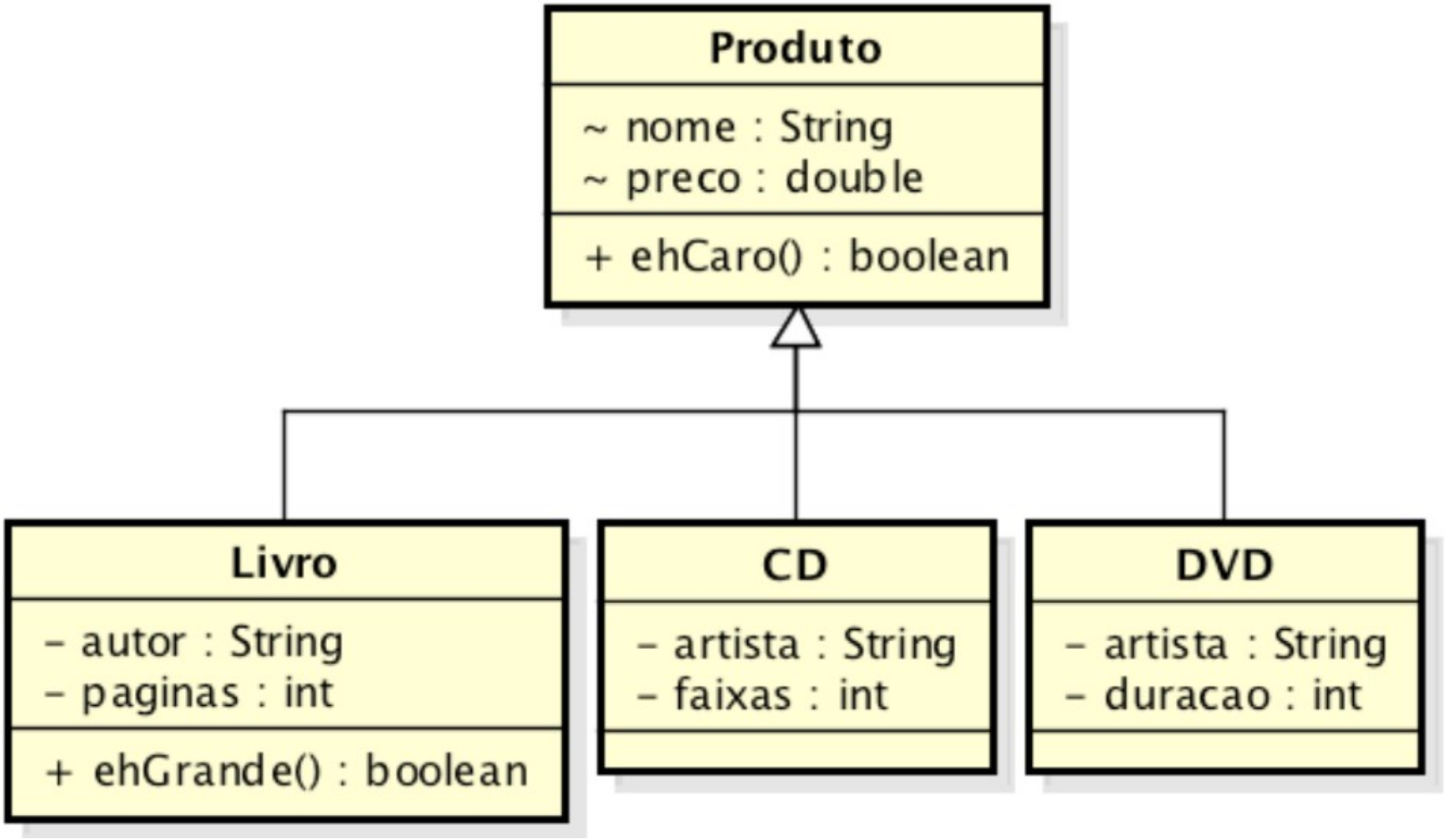
- Superclasse;
- Classe base;
- Classe pai/mãe;
- Classe ancestral, etc.

Livro

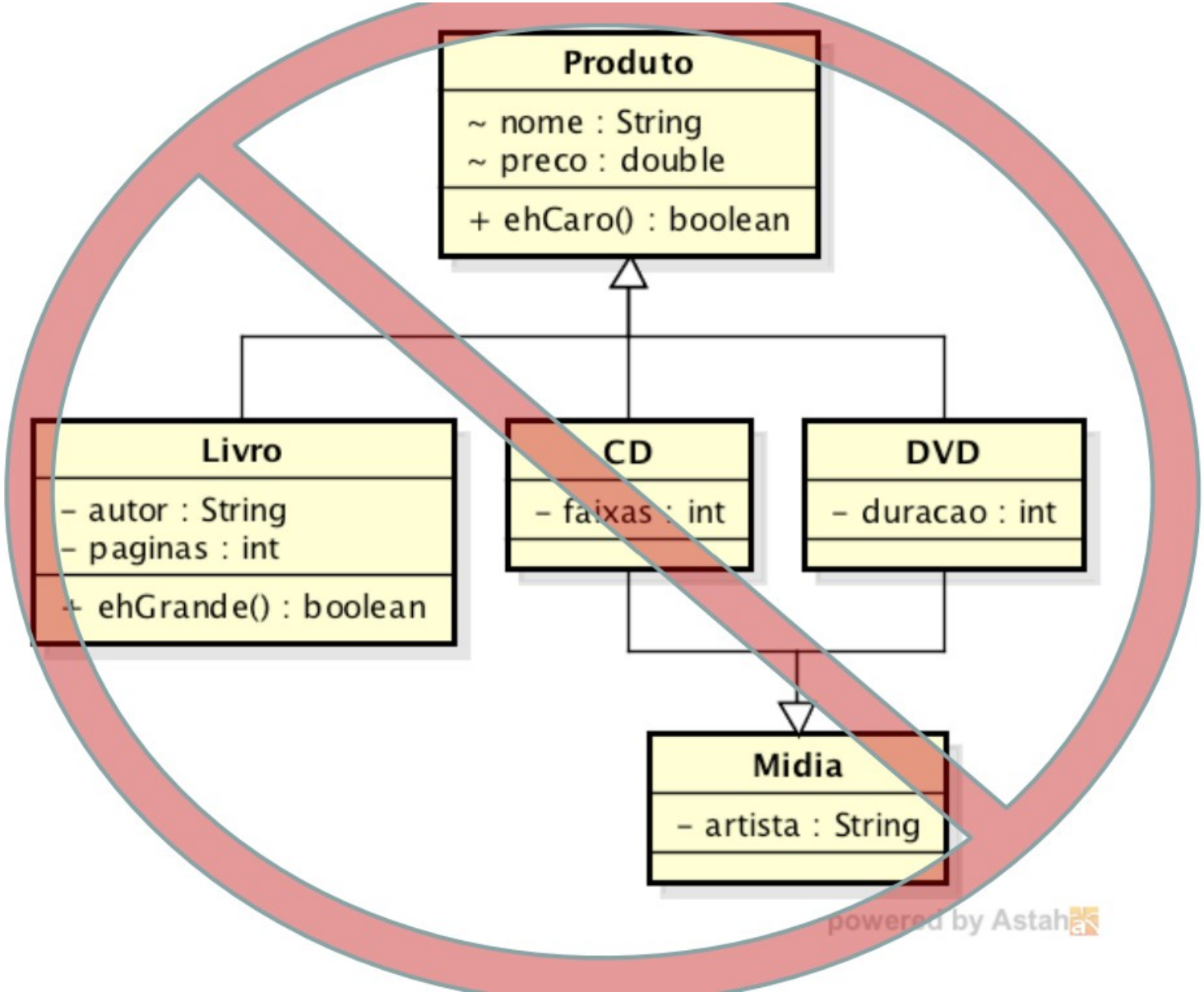
- Subclasse;
- Classe derivada;
- Classe filha;
- Classe descendente, etc.

Java suporta herança simples

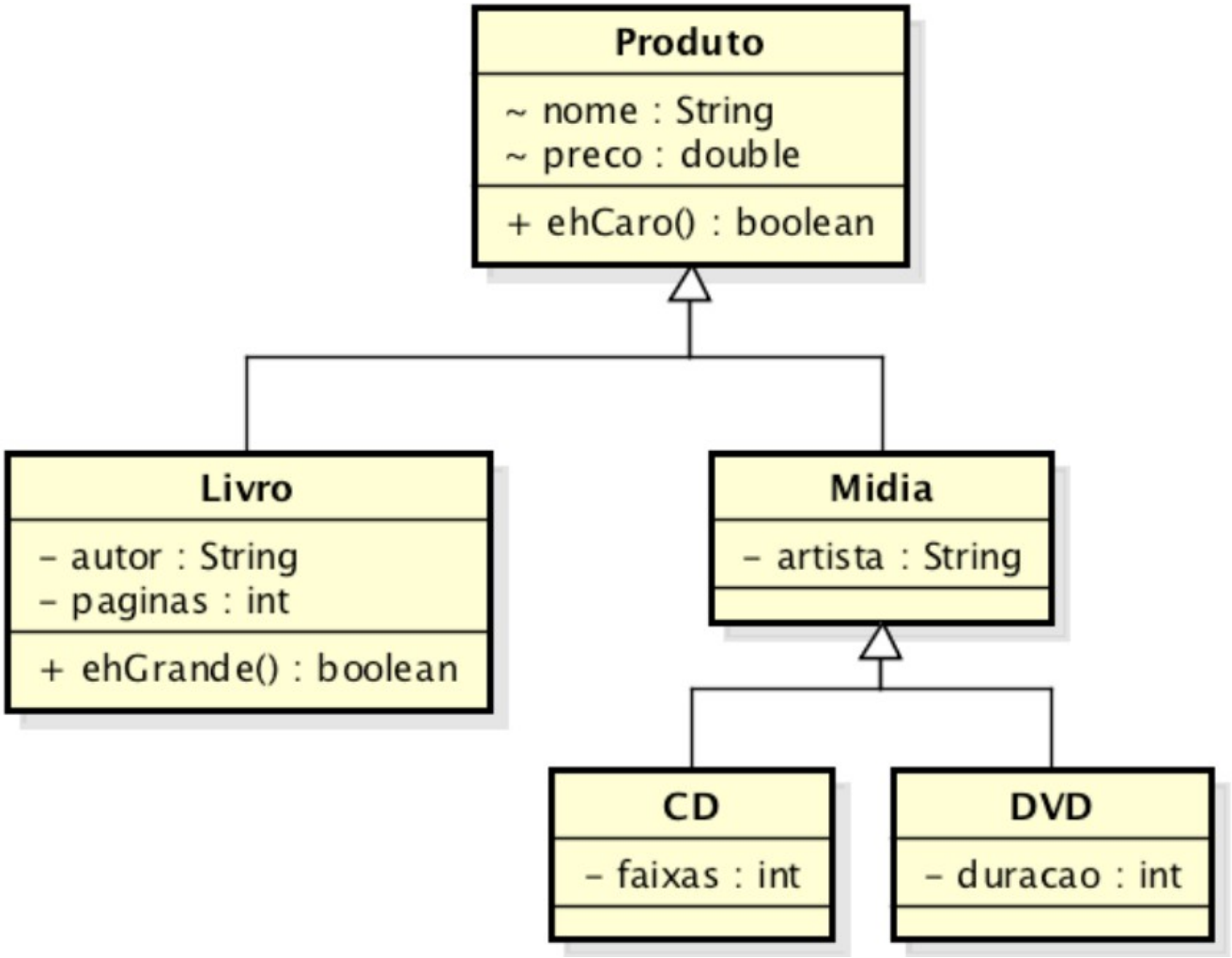
- Uma classe pode ter muitas subclases;
- Uma classe só pode ter uma superclasse.



Java não suporta herança múltipla



Hierarquias de qualquer tamanho



Sintaxe

```
class Subclasse extends Superclasse {
   /* ... */
}
```

- Semântica:
  - A subclasse herda todos os atributos e métodos que a superclasse possuir;
  - Subclasse é uma derivação, um subtipo, uma extensão da superclasse.

Subclasses herdam membros

- Livro possui `autor` e `paginas` (definidos na própria classe);
- Livro possui `nome` e `preco` (definidos na superclasse);
- Livro pode receber mensagens `ehGrande()` (definida na própria classe);
- Livro pode receber mensagens `ehCaro()` (definida na superclasse).

E se `nome` e `preco` fossem definidos como privativos?

Visibilidade dos atributos herdados

- Acesso privativo é só para a própria classe:

```
class Produto {
  private String nome;
  private double preco;

  // Restante da classe...
}

class Livro extends Produto {
  private String autor;
  private int paginas;

  public Livro(String n, double p, String a, int pg) {
     nome = n; preco = p; autor = a; paginas = pg;
  }

  // Res
}
```

```
error: nome has private access in Produto
       nome = n; preco = p; autor = a; paginas = pg;
       ^
```

Visibilidade dos atributos herdados

- Utilizamos o acesso protegido:

```
class Produto {
  protected String nome;
  protected double preco;

  // Restante da classe...
}

class Livro extends Produto {
  private String autor;
  private int paginas;

  public Livro(String n, double p, String a, int pg) {
     nome = n; preco = p; autor = a; paginas = pg;
  }

  // Restante da classe...
}
```

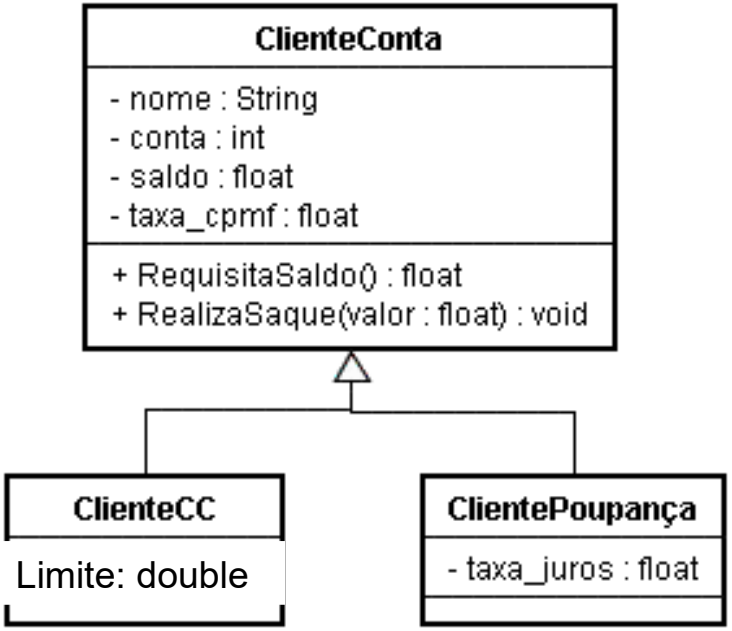
Atenção! O acesso protegido não deveria ser usado automaticamente junto com a herança. Faz sentido a filha acessar atributos da mãe?

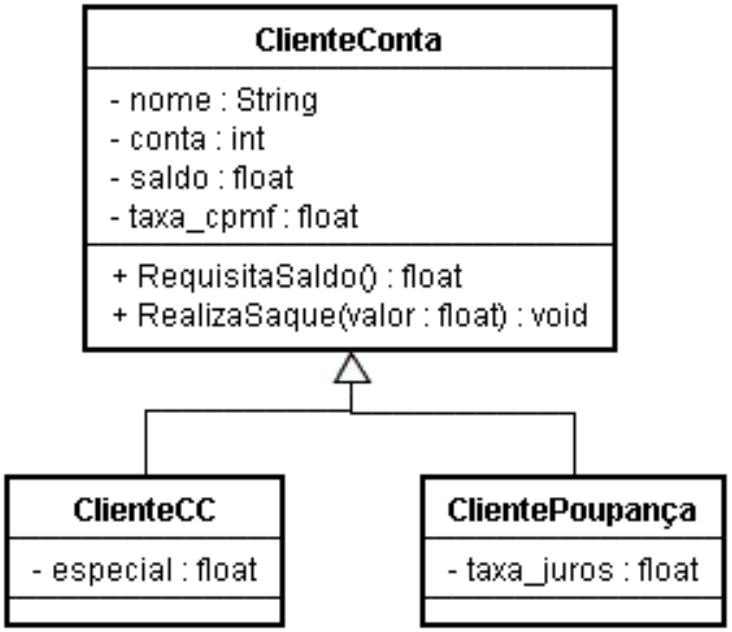
Modificadores de acesso

Acesso	Público	Protegido	Amigo	Privado
A própria classe	Sim	Sim	Sim	Sim
Classe no mesmo pacote	Sim	Sim	Sim	Não
Subclasse em pacote diferente	Sim	Sim	Não	Não
Não-subclasse em pacote diferente	Sim	Não	Não	Não

Exercício

```
public class Conta {
      String nome; int conta;   float saldo;
      static float taxa_cpmf;

      public void sacar (double valor) {
            if (valor <= saldo)
            saldo = saldo – s;
      }
      public float RequisitaSaldo() {
         return saldo;
      }
}
public class ContaCorrente extends Conta {
      double limite;
      public void sacar (double valor) {
      if (valor <= (saldo + especial))
            saldo = saldo – s;
      }
}
public class Poupanca extends Conta {
      static float taxa_juros;
}
```



Exercício

```
public class Conta {
      String nome; int conta;
      double saldo; static float taxa_cpmf;
      public Conta (String pNome,
                    int pConta,
                    float pSaldo) {
            this.nome = pNome;
            this.conta = pConta;
            this. saldo = pSaldo;
      }
}
public class ContaCorrente extends Conta {
     double limite;
     public ContaCorrente (String pNome, int pConta,
              double pSaldo,  double pLimite) {
            super(pNome, pConta, pSaldo);
            this.limite = Plimite;
      }
}
```